

ing to the specific task at hand. Dealing with simulation of a water tank would need a container, or if the system is about a fruits shop with a variety of elements, a store comes to the aid. If you are modelling a machine shop with machines having variable attributes, go for FillerStore. The above is the case of going around the homogenous slots of a resource to get what you need.

Environment is as important as the process

When the very functioning of one's system depends on its interaction with its surroundings, the environment takes precedence. A common base environment is generally used as the norm, performing simulations as fast as possible, while a real-time application demands a separate environment for itself. This could be when the system involves hardware or human interaction. SimPy takes this factor into account and provides separate environments for the two.

Further, the way SimPy works, environment is simply not the factor that encompasses the whole, but it is this simulation environment that manages simulation time, scheduling and processing of events, and takes care of execution. You can flow through this processing in different ways: step through in steps, run for a given time, run till simulation stops on its own or flag the trigger of an event.

A peek-and-step feature lets you

Let us look at how SimPy works with these use-cases for an electric car

#1: Event sequence: The car is parked, charged, waits for the charging to be done and starts driving.

```
>>> class Car(object):
...     def __init__(self, env):
...         self.env = env
...     # Start the run process everytime an instance is created.
...     self.action = env.process(self.run())
...
...     def run(self):
...         while True:
...             print('Start parking and charging at %d'%self.env.now)
...             charge_duration = 5
...             # We yield the process that process() returns
...             # to wait for it to finish
...             yield self.env.process(self.charge(charge_duration))
...
...             # The charge process has finished and
...             # we can start driving again.
...             print('Start driving at %d'%self.env.now)
...             trip_duration = 2
...             yield self.env.timeout(trip_duration)
...
...             def charge(self, duration):
...                 yield self.env.timeout(duration)
```

#2: Many cars go to the charging station to recharge themselves. How is the queue handled?

```
>>> def car(env, name, bcs, driving_time, charge_duration):
...     # Simulate driving to the BCS
...     yield env.timeout(driving_time)
...
...     # Request one of its charging spots
...     print('%s arriving at %d'%(name, env.now))
...     with bcs.request(1) as req:
...         yield req
...
...     # Charge the battery
...     print('%s starting to charge at %s'%(name, env.now))
...     yield env.timeout(charge_duration)
...     print('%s leaving the bcs at %s'%(name, env.now))
```

step through the simulation event-by-event. To integrate the simulation via the graphical user interface, you can repeatedly call a for-loop as below, with repeatedly increasing values, thereby letting you update the process bar according to the pro-

gress of the simulation:

```
for i in range(100):
    env.run(until=i)
    progressbar.update(i)
```

How to check what is happening

It is the basic code that you write in SimPy that decides the entire process at hand. It is then obvious to want to observe how the system is functioning, or maybe to cross-check expected behaviour. With monitoring in SimPy, you can monitor the various processes at hand, how each parameter is handled, the way resources are allocated and used, and trace every event of the simulation.

Monitoring can alert you when a certain event happens or if a certain event does not happen. This could be done at regular intervals or you can even set the monitoring routine to begin if and when something goes wayward, and choose to store the results as a file, a list or directly in the database.

Simply using SimPy

Though it is theoretically possible to perform continuous simulations on SimPy, as a library built for discrete simulation, it does not offer any special features for the same. On the other hand, SimPy is overkill for fixed-size processes that do not interact with each other

or share resources. It is up to you to exploit the built-in features for the right applications and ease the load. **EFY**

Priya Ravindran is M.Sc (electronics) from VIT University, Vellore, Tamil Nadu. She loves to explore new avenues and is passionate about writing